

КОД БЕЗ КОММЕНТАРИЕВ

```
from itertools import *
print(*'1234567')
for per in permutations('ABCDEFGH'):
    p1 = '13 14 16 24 25 36 45 57 67'
    p2 = 'AB AC BC CE AF DG DE DF EG'
    for a, b in zip('1234567', per):
        p1 = p1.replace(a, b)
    for c in p1.split():
        if c not in p2 and c[::-1] not in p2: break
    else: print(*per)
```

=====

КОД С КОММЕНТАРИЯМИ

```
from itertools import * # Импортируем все функции из модуля itertools

print(*'1234567') # Выводим цифры от 1 до 7 через пробел: "1 2 3 4 5 6 7"

for per in permutations('ABCDEFGH'): # Перебираем все возможные перестановки букв A,B,C,D,E,F,G
    p1 = '13 14 16 24 25 36 45 57 67' # Список рёбер графа по таблице (в цифрах)
    p2 = 'AB AC BC CE AF DG DE DF EG' # Список рёбер графа по картинке (в буквах)
    for a, b in zip('1234567', per): # zip('1234567', per) создаёт пары: ('1','A'),('2','B')...
        p1 = p1.replace(a, b)
        # Заменяем цифры (a) в p1 на соответствующие буквы (b) из текущей перестановки
    for c in p1.split(): # p1.split() разбивает строку p1 на отдельные пары (рёбра)
        # Проверяем, все ли рёбра из p1 (после замены) присутствуют в p2
        # проверяем, есть ли ребро (c) или обратное ребро (c[::-1]) в p2
        if c not in p2 and c[::-1] not in p2: break # если нет, то это плохая перестановка, выходим
    else:
        # сюда мы попадем только если не было дострочного выхода из цикла - значит, перестановка хорошая.
        print(*per) # печатаем перестановку букв в распакованном виде - без пробелов, скобок и запятых
```

◆ **ИСХОДНЫЕ ДАННЫЕ**

Цифровой граф (рёбра) : 12 13 14 23 34
Буквенный граф (эталон) : **BA AD AC CB DC**

=====

ПЕРЕБОР ВСЕХ ПЕРЕСТАНОВОК

=====

★ **Перестановка: ABCD**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : AB AC AD BC CD
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✓ ПОДХОДИТ

★ **Перестановка: ABDC**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : AB AD AC BD DC
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✗ НЕ ПОДХОДИТ

★ **Перестановка: ACBD**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : AC AB AD CB BD
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✗ НЕ ПОДХОДИТ

★ **Перестановка: ACDB**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : AC AD AB CD DB
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✗ НЕ ПОДХОДИТ

★ **Перестановка: ADCB**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : AD AC AB DC CB
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✓ ПОДХОДИТ

★ **Перестановка: BDCA**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : BD BC BA DC CA
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✗ НЕ ПОДХОДИТ

★ **Перестановка: CBAD**

Было (цифровой) : 12 13 14 23 34
Стало (после замены) : CB CA CD BA AD
Сравниваем с эталоном : **BA AD AC CB DC**
Результат: ✓ ПОДХОДИТ


```

from itertools import permutations

# --- ИСХОДНЫЕ ДАННЫЕ ---
vertices = '1234'
edges_digit = '12 13 14 23 34'
edges_letter = 'BA AD AC CB DC' # эталон
letters = 'ABCD'

# Преобразуем эталон в множество для сравнения
target_set = set(''.join(sorted(e)) for e in edges_letter.split())

print("◆ ИСХОДНЫЕ ДАННЫЕ")
print(f"    Цифровой граф (рёбра): {edges_digit}")
print(f"    Буквенный граф (эталон): {edges_letter}")
print()
print("=" * 70)
print("ПЕРЕБОР ВСЕХ ПЕРЕСТАНОВОК")
print("=" * 70)

for perm in permutations(letters):
    mapping = {v: perm[i] for i, v in enumerate(vertices)}

    # Заменяем цифры на буквы
    edges_new = edges_digit
    for digit, letter in mapping.items():
        edges_new = edges_new.replace(digit, letter)

    # Преобразуем в множество для сравнения
    new_set = set(''.join(sorted(e)) for e in edges_new.split())
    valid = (new_set == target_set)

    # Вывод в новом формате
    print(f"\n★ Перестановка: {''.join(perm)}")
    print(f"    Было (цифровой): {edges_digit}")
    print(f"    Стало (после замены): {edges_new}")
    print(f"    Сравниваем с эталоном: {edges_letter}")
    print(f"    Результат: {'✓ ПОДХОДИТ' if valid else '✗ НЕ ПОДХОДИТ'}")

```